

Question 1

Fonction est_bissextile():

```
def est_bissextile(annee):
```

```
    """
```

```
    Prend en parametre une annee et renvoie si elle est ou non bissextile
```

```
    :param annee:(int) l'annee dont on verifie la "bissextilite"
```

```
    """
```

```
    assert type(annee)==int,"l'annee doit etre un nombre entier"
```

```
    res = False
```

```
    if annee%4==0 and (annee%400==0 or annee%100!=0):
```

```
        res = True
```

```
    return res
```

Question 2

Fonction determiner_phase():

```
def determiner_phase(jour):
```

```
    """
```

```
    Prend en parametre un entier jour qui correspond a un jour du cycle,
```

```
    et renvoie un entier correspondant a la phase du cycle associee a ce jour
```

```
    :param jour:(int) le jour du cycle dont on veut obtenir la phase associee
```

```
    :cu: jour doit etre entre 1 et 28 inclus
```

```
    """
```

```
assert type(jour)==int,"jour doit etre un entier"
assert jour >= 1 and jour <= 28,"jour doit etre entre 1 et 28 inclus"
```

```
res = None
if jour >= 1 and jour <= 5:
    res = 1
elif jour >= 6 and jour <= 13:
    res = 2
elif jour==14:
    res = 3
else:
    res = 4
return res
```

Question 3

Tests ajoutés à la fonction test_ajouter_jours():

```
assert ajouter_jours((28,1,2025),4) == (1,2,2025) #teste le changement de mois (sur un
mois en 31)

    assert ajouter_jours((30,4,2025),1) == (1,5,2025) #teste le changement de mois (sur un
mois en 30)

    assert ajouter_jours((28, 2, 2024),1) == (29, 2, 2024) #teste le changement de mois en
annee bissextile

    assert ajouter_jours((28, 2, 2023),1) == (1, 3, 2023) #teste le changement de mois en
annee non bissextile

    assert ajouter_jours((27,12,2025),5) == (1,1,2026) #teste le changement d'annee
```

```
assert ajouter_jours((6,7,2025),30) == (5,8,2025) #teste laddition dun mois entier
assert ajouter_jours((6,7,2025),365) == (6,7,2026) #teste laddition dune annee entiere
```

Question 4

Le calendrier renvoyé

BEGIN:VCALENDAR

VERSION:2.0

PRODID:

BEGIN:VEVENT

SUMMARY: Règles

DTSTART:2026312

END:VEVENT

BEGIN:VEVENT

SUMMARY: Règles

DTSTART:202649

END:VEVENT

BEGIN:VEVENT

SUMMARY: Règles

DTSTART:202657

END:VEVENT

END:VCALENDAR,

n'est pas dans un format valide pour deux raisons :

-DTSTART doit être avant SUMMARY (BEGIN, DTSTART, SUMMARY, END pour un évènement dans le calendrier)

-la date n'est pas bien formatée, elle doit faire 8 chiffres, mais vu que les jours et mois passent en paramètres dans l'exemple ne dépassent pas 10 et qu'il n'y a pas de code pour ajouter de 0 devant, il n'y a qu'un seul chiffre pour le jour et qu'un seul chiffre pour le mois.

Pour y remédier :

-inverser le placement de SUMMARY et de DTSTART pour qu'ils soient dans le bon ordre (pour faire cela il faudra aussi déplacer le code qui crée la variable date avant la ligne de DTSTART).

-ajouter du code qui vérifie si jour < 10 pour ajouter un 0 devant le jour, et idem pour mois.

Code modifié de calendrier_cycles():

-inversion de SUMMARY et DTSTART:

```
cal_lignes.append('DTSTART:'+date)
```

```
cal_lignes.append('SUMMARY: Règles')
```

-date qui prend en compte les jours et mois < 10 qui font donc qu'un chiffre au lieu de deux

```
if jour > 9 and mois > 9:
```

```
    date = str(annee)+str(mois)+str(jour)
```

```
elif jour > 9:
```

```
    date = str(annee)+str("0"+str(mois))+str(jour)
```

```
elif mois > 9:
```

```
    date = str(annee)+str(mois)+str("0"+str(jour))
```

```
else:
```

```
    date = str(annee)+str("0"+str(mois))+str("0"+str(jour))
```